CMPUT412 Exercise #4

Antonio Lech Martin-Ozimek

antonio2

March 27, 2025

1. The code visualizes AprilTag detections by first capturing camera images from the Duckiebot, which are then processed through the dt_apriltags detector to identify special fiducial markers. When tags are found, the system extracts the corner coordinates of each tag and draws green lines connecting these points to create a visible bounding box around the tag. At the center of each detected tag, the code displays the tag's unique ID number using OpenCV's text rendering functions. These visual augmentations are overlaid on the original camera feed. The resulting augmented image is then converted back to a compressed format and published to a ROS topic. I commented out that portion of the code, because it slowed down the program.
2.
    a. Convert the incoming RGB camera feed to grayscale since the AprilTag detection algorithm works better with grayscale images
    b. Downsize the image in order to speed up processing
    c. Gaussian Blur the image after in order to smooth out the artifacts
3. My initial rate was about 30 FPS, but I had to down sample it to 10 FPS roughly because the program was slow. Furthermore, there were times when the program itself was running that the bot would stop and while it was stopping, we detected a new tag which would change the most recently detected one and mess up the waiting time. If detecting too frequently, the bot will stop at the stop sign but detect that only a 0.5 second stop is necessary. This is why downsampling is important.
4. Detecting apriltags as I described requires the image to be converted to grayscale and then there is a distortion matrix that needs to be applied to remove the distortion. After that there is some noise removal processes that can be run and few other options. But that is all done internally in the dt_apriltags library so really all we do is manipulate the object returned by the detection algorithm.
5. As I described earlier, the intersection detection would change right as the bot stopped if it was too fast and then we would wait for the wrong period of time. I had

some difficulty here because the bot stopped moving in a straight line for some reason after the first intersection. I recalibrated it and it still did not work properly.

6. For dealing with double lines it was quite simple, I just detected the two largest blue shapes from my masks and checked that the distance between their centers was a fixed amount which I measure to be about 0.05m according to my distance formula in my program. That fixed amount meant that I could determine when two blue lines were found and then I would calculate the distance to the closer one. If there was only one line in view, the car would just keep moving forward.
7. For this section, it was very simple. I used a state machine, which was recommended by chatgpt and it gave me the template for the state machine. Essentially, it was looking for a double blue line. If it found one it would stop for an appropriate amount of time, then move forward looking for the red line. Because my red bounds were so large, the duckies were visible using it. So I just used my red line detection. And if the state machine found a red line it would just wait until it was gone. Meaning that as long as a duckie was on the screen it would wait. After the duckies were off its screen, I still made it wait a bit longer to hypothetically allow the duckies to finish crossing. Then it went back to looking for blue lines. So it would see the double and wait there again. So it was like a double wait time compared to the first crossing. However, for the safety of the ducks, I thought it was necessary to build redundancy into the system.
   My method failed sometimes when the ducks blocked a portion of the second line and my robot got too close. It would see the second blue line as a double and attempt to approach it through the ducks. Furthermore, it would fail if it missed the first line and already moved too close to the ducks. Then it would just keep driving. Aside from that, it worked very well.

8. The method I used to detect the bot was the same as detecting blue lines. I looked for a large blue shape of an area of about 1000 pixels minimum. When that was found, I check the distance using my distance formula and ensured that I was about 0.3 meters away when I stopped. This worked very well, but only when the bot was not on blue lines. I specifically made sure of that but never tested in an environment where the robot could easily fail. I could not get a video of the robot with my phone because it died and I had to get a ride. The video I took was from the perspective of the robot and you can see it maneuvering around the broken down robot.

   I did not try other methods for this situation because I was very low on time. Another method I could have tried was to detect the shape of the back of the car instead of

just the size. This would make my algorithm more robust to other sources of blue colour.

9. I did not manage to get my lane following working in the last lab. So to maneuver around the robot, I used sequential hardcoded movements. Similar to someone who is trying to parallel park and they follow visual cues. I rotated the bot 45 degrees then merged into the other lane. Drove straight to pass, then merged back in before driving a little straight again.
10. I could not get the lane following to work, because the edge cases were very difficult to deal with. When turning the middle line disappears, so I have to default to just using the white line which means I need a whole new algorithm essentially. When estimating the middle line, if my car is turning horizontally I need a new algorithm for estimating the midline. Finally, my bot does not have a way to localize itself. I can see an image and estimate the midline, but I do not know where that midline is relative to my car, just the image taken by my car. So I struggle in lane following accurately. I could not get it implemented for this lab. But I am working on it for the final project.

Overall this lab was a bit more straightforward than exercise #3. I think the colour detection and masking it simpler to implement than lane following with a PID controller. I do however, feel as though there should be three exercises total instead of 4. And that maybe, it would allow for better final projects.